

SQL Explanation for Admin_Aggregation_View.sql

SQL Code

```
-- =====
-- AGGREGATION FUNCTIONS VIEW
-- =====
-- DBMS Concept: Aggregation Functions (COUNT, SUM, AVG, MAX, MIN)
-- Purpose: Provide a unified view of user workout statistics
-- =====

CREATE OR REPLACE VIEW view_set_combined AS
SELECT
    up.user_id,
    up.name as user_name,
    up.email,
    -- COUNT aggregations
    COUNT(DISTINCT tr.record_id) as total_workouts,
    COUNT(DISTINCT tr.exercise_id) as unique_exercises,
    COUNT(DISTINCT tr.date) as unique_workout_dates,
    -- SUM aggregations
    COALESCE(SUM(tr.sets), 0) as total_sets,
    COALESCE(SUM(tr.reps), 0) as total_reps,
    COALESCE(SUM(tr.duration_minutes), 0) as total_duration_minutes,
    COALESCE(SUM(tr.weight_used * tr.sets), 0) as total_volume,
    -- AVG aggregations
    ROUND(AVG(tr.sets), 2) as avg_sets,
    ROUND(AVG(tr.reps), 2) as avg_reps,
    ROUND(AVG(tr.weight_used), 2) as avg_weight,
    ROUND(AVG(tr.duration_minutes), 2) as avg_duration,
    -- MIN aggregations
    MIN(tr.weight_used) as min_weight,
    MIN(tr.sets) as min_sets,
    MIN(tr.reps) as min_reps,
    MIN(tr.date) as first_workout_date,
    -- MAX aggregations
    MAX(tr.weight_used) as max_weight,
    MAX(tr.sets) as max_sets,
    MAX(tr.reps) as max_reps,
    MAX(tr.date) as last_workout_date
FROM user_profile up
LEFT JOIN training_records tr ON up.user_id = tr.user_id
GROUP BY up.user_id, up.name, up.email
```

```
HAVING COUNT(tr.record_id) > 0
ORDER BY total_workouts DESC, user_name;
```

What is this file for?

This SQL file creates a **VIEW** to combine user information with detailed workout statistics using **aggregation functions**, **joins**, and **COALESCE** to avoid NULL values.

Line-by-line Explanation

CREATE VIEW

Creates a virtual table named `view_set_combined`. If the view already exists, it gets replaced.

Selecting User Information

The query selects user details such as `user_id`, `name`, and `email` from the `user_profile` table.

Table Aliases (`up` and `tr`)

- `up` is an alias for the `user_profile` table.
 - `tr` is an alias for the `training_records` table.
 - `up.user_id` means we are accessing the `user_id` column from the `user_profile` (alias `up`) table.
-

Aggregation Functions

COUNT()

Used to count distinct workouts, exercises, and workout dates.

SUM() with COALESCE()

`COALESCE(SUM(tr.sets), 0)` ensures that if `SUM(tr.sets)` returns NULL (for users who have no workout records), it shows **0** instead of NULL.

COALESCE returns the first non-NULL value from the list.

AVG() and ROUND()

`AVG()` calculates average values. `ROUND(value, 2)` rounds the results to 2 decimal places.

MIN() and MAX()

These find the smallest or largest values for weight, sets, reps, and dates.

JOIN Used

`LEFT JOIN` is used to include all users even if they have no workout records.

GROUP BY

Groups all rows based on each unique user so that aggregates are calculated correctly per user.

HAVING

Filters groups after aggregation. Here it ensures only users with at least one workout (`COUNT(tr.record_id) > 0`) appear.

ORDER BY

Sorts the results by total workouts (descending) and then by user name.